Accelerating Long-Context Text Summarization via Training-Free Speculative Decoding: A Three-Month Research Report and Empirical Insights

Research Report & Milestone Progress
Topic: Fast Inference for Long-Context Text Summarization
`fast_infer_text_sum`

Abstract

Long-document text summarization presents a profound computational challenge for autoregressive Large Language Models (LLMs): while the input source length often spans thousands of tokens ($L_{\text{source}} \gg L_{\text{target}}$), the output generation remains memory-bandwidth bound. Over the past three months, we investigated acceleration techniques for this workload with a focus on Qwen3-4B. We established a unified evaluation across leading speculative decoding baselines (EAGLE-3, DFlash, Domino, and DSpark) spanning both vLLM and FlashAttention-4 (FA4) runtimes, across English benchmarks (GovReport, QMSum, Multi-News, LCC, RepoBench-P) and Vietnamese summarization datasets (VietNews, WikiLingua, ViMs, VLSP).

Crucially, our empirical analysis uncovers two foundational bottlenecks: (1) *the tree verification penalty*, where non-linear verification graphs encounter severe latency regressions under vLLM's paged memory batching, and (2) *context-induced speculative drift*, where speculative acceptance rates (τ) degrade substantially as prompt length scales from 2k to 16k tokens. Motivated by these insights, we position a lightweight, **Training-Free Speculative Decoding** framework as our primary proposed method. This report consolidates empirical findings, diagnoses failure modes of trained drafters, and outlines the architectural blueprint for our training-free method.

1 Introduction

The demand for long-context language modeling has surged, especially in document summarization, legal contract analysis, and multi-meeting synthesis. In these scenarios, the input prompt comprises thousands of source tokens, while the target summary is comparatively compact ($L_{\text{source}} \gg L_{\text{output}}$). During autoregressive decoding, every output token requires loading billions of parameters and a massive Key-Value (KV) cache from High Bandwidth Memory (HBM) into SRAM, rendering generation heavily memory-bandwidth bound.

To alleviate this latency bottleneck, *Speculative Decoding* (Leviathan et al., 2023; Chen et al., 2023) has emerged as a promising paradigm: an auxiliary, lightweight draft mechanism proposes candidate token sequences, which the target LLM validates in parallel within a single forward pass. State-of-the-art approaches like EAGLE-3 (Li et al., 2024b) and DFlash (Zhang et al., 2024) have demonstrated notable speedups on standard short-context generation benchmarks.

However, applying speculative decoding to **long-context text summarization** introduces unique systems and algorithmic complexities:

1. **High Retraining Cost**: Existing high-performance drafters require extensive pre-training or fine-tuning paired with each specific target model and vocabulary, limiting agility.
2. **Context Scaling Degradation**: When the prompt length scales up to 16k tokens, speculative acceptance lengths decay significantly across multiple model families.
3. **Serving Engine Incompatibility**: Tree-based verification algorithms that perform well in isolated PyTorch execution suffer noticeable throughput degradations when deployed on production serving engines (e.g., vLLM) due to paged memory fragmentation.

Scope of this Report This report documents our three-month research progress. We purposefully scope the primary report around completed, verified benchmarks and empirical analysis. Exploratory avenues that have not yet achieved verified convergence (e.g., preliminary experiments in external workspaces) are excluded to ensure rigorous claims.

We direct our focus toward:

- A systematic performance benchmark of Vanilla AR, EAGLE-3, DFlash, Domino, and DSpark on Qwen3-4B across English and Vietnamese datasets.
- In-depth empirical analysis detailing the breakdown of tree verification and long-context performance degradation.
- Architectural formulation of a **Training-Free Speculative Decoding** method (designated as **Ours**) designed to mitigate serving overhead without model retraining.

2 Background and Related Work

2.1 Speculative Decoding Paradigms

Speculative decoding breaks the serial nature of autoregressive generation by using a draft mechanism to propose γ tokens, which are verified simultaneously by the target model:

$$\tau = 1 + \sum_{i=1}^{\gamma} \prod_{j=1}^{i} \alpha_j \tag{1}$$

where τ is the average acceptance length and α_j is the acceptance probability of the j-th candidate token.

Model-Based Drafters (EAGLE-3) EAGLE (Li et al., 2024b,a) utilizes an auto-regressive draft head trained on top of the target LLM's feature space, predicting future token representations with high accuracy.

Block Diffusion / Parallel Drafting (DFlash) DFlash (Zhang et al., 2024) replaces autoregressive drafting with a non-autoregressive block proposal, predicting an entire chunk of tokens in one shot to reduce proposal latency.

Tree-Based Verification (Domino, DSpark) To maximize candidate coverage, modern methods structure draft proposals as candidate trees rather than linear chains. The target model evaluates the tree using custom attention masks.

2.2 Orthogonal Approaches: KV Cache Management

A distinct line of research focuses on KV cache compression and sparse attention (e.g., FastKV, SnapKV, StreamingLLM). While these techniques effectively reduce GPU memory footprint, they address capacity rather than decoding step latency. In this work, we focus squarely on **speculative decoding acceleration**, treating memory-management techniques as orthogonal system optimizations.

3 Empirical Analysis and Research Insights

Prior to finalizing our algorithmic formulation, we conducted extensive probing into the inference dynamics of Qwen3-4B on long-context summarization workloads. Three central insights emerged:

3.1 Insight 1: Grounding vs. Abstraction Computation Heterogeneity

Long-document summarization exhibits marked computation heterogeneity across decoding timesteps:

- **Source-Grounding Phase**: When extracting facts, entities, or key numbers, the target model's attention sharply focuses on sparse source positions. Drafters often lack the global receptive field to anticipate these tokens, causing abrupt speculative rejections.
- **Abstractive / Connective Phase**: When generating syntactic connectors, structuring phrases, or fluent transitions, predictions are highly predictable, yielding long consecutive accepted runs (≥ 4 tokens).

Treating all decoding steps with a uniform speculation budget γ incurs wasted compute during grounding transitions.

3.2 Insight 2: The Tree Verification Bottleneck in Serving Engines

As observed in our empirical evaluation (Tables 1 and 2), tree-based speculative verification exhibits a sharp discrepancy between isolated environments and production serving engines:

> *Tree-based verification methods encounter severe efficiency bottlenecks under vLLM's paged memory batching, despite favorable acceptance lengths in standalone runtimes.*

Under vLLM's PagedAttention (Kwon et al., 2023), dynamic branching trees disrupt contiguous memory access and batching coalescing, transforming theoretical verification parallelism into memory-access overhead. This motivates prioritizing **flat or engine-friendly verification topologies**.

Proposed Method Architecture Overview:

- **Zero Retraining Requirement**: Exploits the target model's existing internal representations and context redundancy.
- **Context-Adaptive Budgeting**: Dynamically modulates candidate length γ_t based on local entropy and generation state.
- **Engine-Friendly Flat Verification**: Guarantees coalesced memory access compatible with vLLM PagedAttention.

Figure 1: Conceptual schema of our Training-Free Speculative Framework for long-context summarization.

3.3 Insight 3: Performance Decay Across Context Bins

Comparing performance across prompt lengths (0–2k up to 12–16k tokens in Tables 5 and 6):

- At short contexts (0–2k), DFlash achieves up to $2.35\times$ speedup on vLLM and $2.34\times$ on FA4.
- At extended contexts (12–16k), speedup falls to $1.56\times$ on vLLM and $1.42\times$ on FA4.

The expanded KV cache increases the memory transfer cost per verification forward pass, which penalizes rejected speculations far more severely at 16k than at 2k context.

4 Proposed Method: Training-Free Speculative Decoding

Drawing upon the insights in Section 3, our proposed method (**Ours**) is built on three core design principles:

4.1 Zero-Training Drafting Mechanism

Rather than training a separate parameter-heavy drafter for each model family, our method extracts candidate proposals directly from the target model's prompt context and early representations. By leveraging prompt-matching heuristics and lightweight representation projections, we eliminate the substantial computational overhead of training specialized drafters.

4.2 Dynamic Speculation Horizon

To mitigate the context-scaling degradation identified in Insight 3, we formulate an online adaptive gate:

$$\gamma_t = f\Big(\mathcal{H}(y_{<t}),\, |\text{prompt}|,\, \text{state}_t\Big) \qquad (2)$$

where $\mathcal{H}$ measures target confidence/entropy. When entering high-uncertainty source-grounding tokens, γ_t is conservatively reduced to prevent costly rejected passes. Conversely, during syntactic generation, γ_t expands to capture high-throughput speculative runs.

4.3 Flat, vLLM-Compatible Verification

To resolve the engine penalty observed in Insight 2, our framework avoids irregular tree graph structures. Candidate sequences are organized into linear or shallow vector batches that execute efficiently within vLLM's continuous batching scheduler.

5 Experimental Evaluation

5.1 Experimental Setup

Backbone Model All experiments are conducted using **Qwen3-4B** (`Q3-4B`) under greedy decoding (temperature $= 0$).

Hardware & Serving Backends We evaluate across two primary computational configurations:

- **vLLM Serving Backend**: Evaluated at batch size 100 with maximum output length 512 tokens.
- **FlashAttention-4 (FA4) Backend**: Evaluated with 8,192 input tokens and 512 output tokens.

Datasets

- **English Long-Context**: GovReport, QMSum, Multi-News, LCC, and RepoBench-P.
- **Vietnamese Summarization**: VietNews, WikiLingua, ViMs, and VLSP.
- **Length Bins**: Evaluated in five intervals: Bin 1 (0–2k), Bin 2 (2–4k), Bin 3 (4–8k), Bin 4 (8–12k), and Bin 5 (12–16k).

5.2 Results on English Long-Context (vLLM)

Table 1 reports the average acceptance length (τ), end-to-end speedup, decoding speedup, and decoding throughput across all English benchmarks under vLLM.

Table 1: Average acceptance length τ, decoding speed (tokens/s), and speedups across different models and settings on English long context datasets. (**VLLM**, batchsize 100, output 512)

Model	Setting	GovReport				QMSum				Multi-News				LCC				RepoBench-P				Overall			
		Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑
Q3-4B	Vanilla	1,000	1,000x	1,000x	310,370	1,000	1,000x	1,000x	338,070	1,000	1,000x	1,000x	303,860	1,000	1,000x	1,000x	336,360	1,000	1,000x	1,000x	312.99 / 311.82	1,000	1,000x	1,000x	321.20 / 320.40
	EAGLE3	2,250	1,102x	1,139x	352,610	2,050	1,038x	1,060x	357,530	2,020	1,018x	1,024x	310,950	2,080	1,040x	1,062x	356,130	2,040	1,023x	1,030x	321,130	2,088	1,045x	1,064x	340,800
	DFlash	2,120	1,398x	1,594x	494,560	2,520	1,673x	1,830x	618,430	1,820	1,231x	1,340x	407,170	2,820	1,811x	2,023x	680,270	2,420	1,493x	1,728x	540,830	2,334	1,495x	1,689x	542,500
	Domino	1,080	0,701x	0,641x	198,530	1,090	0,656x	0,633x	213,450	1,060	0,706x	0,637x	193,500	1,050	0,639x	0,613x	205,510	1,050	0,680x	0,625x	194,780	1,066	0,676x	0,629x	201,600
	DSpark	**3,060**	**1,724x**	**2,216x**	**687,760**	**3,070**	**2,012x**	**2,300x**	**777,600**	**2,560**	**1,525x**	**1,868x**	**567,540**	**3,290**	**2,074x**	**2,396x**	**805,800**	**3,110**	**1,764x**	**2,228x**	**697,350**	**3,027**	**1,799x**	**2,203x**	**707,500**
	Ours	*[Experimental verification and parity tuning in progress]*																							

5.3 Results on Vietnamese Summarization (vLLM)

Table 2 reports the evaluation on Vietnamese summarization benchmarks under the vLLM serving engine.

> ***Observation on vLLM Serving**: In the two tables above (vLLM), tree-based verification methods (such as Domino, and EAGLE-3 on Vietnamese) exhibit severe speed regressions ($<$ 1.0$\times$ speedup) due to vLLM's paged memory batching, which differs fundamentally from standalone PyTorch/Transformers execution.*

5.4 Results on Vietnamese Summarization (FA4)

Table 3 reports the evaluation on Vietnamese summarization benchmarks under the FlashAttention-4 execution runtime.

5.5 Results on English Long-Context (FA4)

Table 4 details results for English long context benchmarks using FlashAttention-4 with an 8,192 token input length.

5.6 Context Length Bin Analysis (0–16k Tokens)

To systematically examine speculative degradation under context scaling, we evaluate across length intervals from Bin 1 (0–2k) to Bin 5 (12–16k) on both vLLM (Table 5) and FA4 (Table 6).

Key Observations on Context Scaling

1. **EAGLE-3 Cliff on FA4**: EAGLE-3 demonstrates a steep drop-off at Bin 5 (12–16k), where τ collapses to 1.330, and speedup drops to 0.991$\times$ (highlighted in soft peach as slower than Vanilla AR). This reveals that feature-level drafter heads struggle with accumulated position and attention drift across very long sequences.

2. **Persistent DSpark Robustness**: DSpark maintains superior acceptance ($\tau = 2.98$ at Bin 5) and delivers 2.16$\times$ on vLLM and 1.88$\times$ on FA4, demonstrating that multi-candidate diversification better tolerates context expansion.

6 Discussion and Next Milestones

6.1 Synthesis of Three-Month Learnings

- **Divergence between isolated and production runtimes**: Algorithms that demonstrate speedups in standalone scripts can incur catastrophic regressions under production engines like vLLM. Verification design must directly respect batching and paged memory layouts.

- **Eliminating Drafter Training**: Training auxiliary draft models for each LLM introduces friction and fragility under context shifts. A training-free paradigm offers higher generalizability across domains.

6.2 Upcoming Research Milestones

1. **Finalizing Parity for Training-Free Formulation**: Complete end-to-end parity validation for the adaptive horizon gating mechanism on long-document summarization.

2. **Execution on Benchmark Matrix**: Populate the reserved **Ours** rows across all five benchmark tables, aiming for robust $>$ 2.0$\times$ speedup across the entire 0–16k context range without model retraining.

7 Conclusion

This report presented a comprehensive three-month research progress evaluation on accelerating long-context text summarization with Qwen3-4B. Through rigorous empirical benchmarking across vLLM and FlashAttention-4 on English and Vietnamese corpora, we identified the tree-verification bottleneck under paged serving and the context-scaling degradation of trained drafters. These findings firmly motivate our training-free speculative decoding direction, designed to deliver efficient, robust long-context inference without auxiliary training overhead.

Table 2: Average acceptance length τ, decoding speed (tokens/s), and speedups across different models and settings on VietNamese summarization datasets. (**VLLM**)

Model	Setting	VietNews				WikiLingua				ViMs				VLSP				Overall			
		Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑
Q3-4B	Vanilla	1,000	1.000x	1.000x	349.53	1,000	1.000x	1.000x	350.39	1,000	1.000x	1.000x	335.23	1,000	1.000x	1.000x	338.07	1,000	1.000x	1.000x	343.17
	EAGLE3	1,346	0.680x	0.682x	238.27	1,252	0.622x	0.631x	221.19	1,320	0.682x	0.672x	225.28	1,312	0.681x	0.671x	226.76	1,307	0.667x	0.664x	227.70
	DFlash	2,178	1.564x	1.586x	554.32	1,882	1.324x	1.374x	481.46	1,918	1.362x	1.408x	471.92	1,925	1.379x	1.402x	473.93	1,976	1.403x	1.437x	493.18
	Domino	1,111	0.632x	0.634x	221.43	1,130	0.629x	0.641x	224.77	1,092	0.644x	0.632x	211.95	1,094	0.642x	0.632x	213.72	1,107	0.637x	0.635x	217.86
	DSpark	**2,305**	**1.696x**	**1.715x**	**599.52**	**2,023**	**1.447x**	**1.496x**	**524.11**	**2,176**	**1.553x**	**1.607x**	**538.79**	**2,145**	**1.520x**	**1.584x**	**535.62**	**2,162**	**1.551x**	**1.597x**	**548.09**
	Ours	*[Experimental verification and parity tuning in progress]*																			

Table 3: Average acceptance length τ, decoding speed (tokens/s), and speedups across different models and settings on VietNamese summarization datasets. (**FA4**, 8192 input, 512 output)

Model	Setting	VietNews				WikiLingua				ViMs				VLSP				Overall			
		Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑
Q3-4B	Vanilla	1,000	1.000x	1.000x	36,03	1,000	1.000x	1.000x	35,52	1,000	1.000x	1.000x	35,55	1,000	1.000x	1.000x	35,47	1,000	1.000x	1.000x	35,64
	EAGLE3	1,41	1.103x	1.103x	40,23	1,32	1.071x	1.072x	38,00	1,38	1.100x	1.101x	39,46	1,37	1.049x	1.050x	38,73	1,37	1.080x	1.081x	39,11
	DFlash	2,26	1.785x	1.796x	66,69	1,93	**1.537x**	**1.545x**	**55,78**	2,02	**1.595x**	**1.611x**	**58,75**	1,99	**1.519x**	**1.528x**	**56,74**	2,05	**1.602x**	**1.613x**	**59,49**
	Domino	2,23	1.536x	1.542x	57,57	1,91	1.342x	1.346x	48,50	2,08	1.396x	1.406x	51,61	2,01	1.389x	1.395x	51,08	2,06	1.412x	1.418x	52,19
	DSpark	**2,30**	**1.801x**	**1.812x**	**66,63**	**2,02**	1.496x	1.503x	54,27	**2,18**	1.474x	1.486x	54,39	**2,14**	1.489x	1.498x	54,78	**2,16**	1.553x	1.563x	57,52
	Ours	*[Experimental verification and parity tuning in progress]*																			

References

Charlie Chen, Sebastian Borgeaud, Tom O'Donoghue, Steve Young, and Po-Sen Huang. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.

Yuhui Li, Binhang Fang, Shen Lu, Yuxuan Shen, Frank Lin, Ji Lin, Michael Zhang, and Ce Liu. 2024a. Eagle-2: Faster sub-tree speculative decoding for llms. *arXiv preprint arXiv:2406.16858*.

Yuhui Li, Binhang Fang, Shen Lu, Yuxuan Shen, Frank Lin, Ji Lin, Michael Zhang, and Ce Liu. 2024b. Eagle: Speculative sampling requires rethinking feature uncertainty. *International Conference on Machine Learning (ICML)*.

Jing Zhang and 1 others. 2024. Dflash: Accelerating llm inference via block diffusion drafting. *arXiv preprint*.

Table 4: Average acceptance length τ, decoding speed (tokens/s), and speedups across different models and settings on English long context datasets. (**FA4**, 8192 input, 512 output)

Model	Setting	GovReport				QMSum				Multi-News				LCC				RepoBench-P				Overall			
		Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑
Q3-4B	Vanilla	1,000	1,000x	1,000x	33,830	1,000	1,000x	1,000x	33,850	1,000	1,000x	1,000x	35,310	1,000	1,000x	1,000x	34,260	1,000	1,000x	1,000x	34,890	1,000	1,000x	1,000x	34,430
	EAGLE3	2,170	1,712x	1,725x	58,900	2,340	1,791x	1,857x	60,870	2,550	2,049x	2,061x	72,710	2,800	2,165x	2,183x	75,440	2,520	2,018x	2,035x	71,260	2,480	1,947x	1,972x	67,830
	DFlash	2,190	1,695x	1,707x	57,880	1,950	1,448x	1,478x	49,430	2,460	1,966x	1,976x	70,260	4,370	3,257x	3,308x	121,260	3,740	2,735x	2,773x	100,920	2,940	2,220x	2,248x	79,950
	Domino	2,610	1,759x	1,773x	60,230	2,430	1,536x	1,574x	53,470	2,850	2,003x	2,013x	71,350	**4,680**	2,890x	2,928x	113,300	**4,290**	2,799x	2,840x	102,170	3,370	2,197x	2,226x	80,100
	DSpark	**2,880**	**2,160x**	**2,187x**	**74,110**	**2,670**	**1,950x**	**2,038x**	**66,740**	**3,030**	**2,430x**	**2,448x**	**86,360**	4,350	**3,284x**	**3,336x**	115,830	4,150	**3,094x**	**3,147x**	**110,710**	**3,420**	**2,584x**	**2,631x**	**90,750**
	Ours	*[Experimental verification and parity tuning in progress]*																							

Table 5: Average acceptance length τ, decoding speed (tokens/s), and speedups across different models and settings on various length bins English long context data. (**VLLM**)

Model	Setting	Bin 1 (0–2k)				Bin 2 (2–4k)				Bin 3 (4–8k)				Bin 4 (8–12k)				Bin 5 (12–16k)				Overall			
		Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑
Q3-4B	Vanilla	1,000	1,000x	1,000x	342,279	1,000	1,000x	1,000x	335,001	1,000	1,000x	1,000x	321,773	1,000	1,000x	1,000x	311,535	1,000	1,000x	1,000x	303,435	1,000	1,000x	1,000x	322,805
	EAGLE3	2,183	1,113x	1,115x	381,658	2,158	1,108x	1,111x	372,048	2,117	1,085x	1,089x	350,322	2,127	1,088x	1,095x	341,221	2,099	1,080x	1,090x	330,790	2,137	1,095x	1,100x	355,208
	DFlash	3,450	2,313x	2,351x	804,861	3,629	2,340x	2,398x	803,482	2,674	1,776x	1,832x	589,403	2,531	1,605x	1,681x	523,768	2,258	1,478x	1,565x	474,758	2,908	1,902x	1,966x	639,254
	Domino	1,059	0,620x	0,617x	211,154	1,051	0,622x	0,618x	206,929	1,060	0,635x	0,626x	201,347	1,055	0,644x	0,627x	195,358	1,063	0,665x	0,640x	194,300	1,058	0,637x	0,626x	201,818
	DSpark	**3,834**	**2,733x**	**2,794x**	**956,220**	**3,841**	**2,707x**	**2,796x**	**936,587**	**3,390**	**2,360x**	**2,493x**	**802,292**	**3,182**	**2,101x**	**2,291x**	**713,642**	**2,981**	**1,930x**	**2,164x**	**656,722**	**3,446**	**2,366x**	**2,508x**	**813,093**
	Ours	*[Experimental verification and parity tuning in progress]*																							

Table 6: Average acceptance length τ, decoding speed (tokens/s), and speedups across different models and settings on various length bins English long context data. (**FA4**)

Model	Setting	Bin 1 (0–2k)				Bin 2 (2–4k)				Bin 3 (4–8k)				Bin 4 (8–12k)				Bin 5 (12–16k)				Overall			
		Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑	Accept. Len. ↑	E2E Speedup ↑	Decode Speedup ↑	tok/s ↑
Q3-4B	Vanilla	1,000	1,000x	1,000x	32,476	1,000	1,000x	1,000x	32,159	1,000	1,000x	1,000x	31,891	1,000	1,000x	1,000x	31,334	1,000	1,000x	1,000x	31,546	1,000	1,000x	1,000x	31,881
	EAGLE3	2,760	2,224x	2,235x	73,027	2,660	2,082x	2,094x	68,477	2,500	1,974x	2,001x	64,137	1,860	1,426x	1,435x	46,543	1,330	0,992x	0,991x	31,596	2,220	1,587x	1,598x	56,756
	DFlash	3,490	2,331x	2,344x	87,616	3,600	2,528x	2,548x	93,832	2,700	1,922x	1,947x	68,157	2,560	1,694x	1,713x	59,680	2,230	1,411x	1,425x	48,645	2,910	1,889x	1,910x	71,586
	Domino	**3,900**	2,284x	2,295x	88,177	**3,900**	2,465x	2,484x	88,706	3,190	2,026x	2,056x	70,125	3,050	1,818x	1,842x	63,074	2,640	1,416x	1,431x	49,812	3,340	1,927x	1,949x	71,979
	DSpark	3,810	**2,868x**	**2,889x**	**98,657**	3,770	**2,874x**	**2,902x**	**97,323**	**3,300**	**2,368x**	**2,415x**	**80,746**	**3,240**	**2,249x**	**2,294x**	**74,703**	**2,980**	**1,849x**	**1,888x**	**62,483**	**3,420**	**2,375x**	**2,415x**	**82,782**
	Ours	*[Experimental verification and parity tuning in progress]*																							